

Spring的核心概念

2026年3月1日 11:16

IoC(Inversion of Control): 控制反转, 由主动new产生对象转换为由外部提供Bean (IoC容器创建的对象)

DI(Dependency Injection): 依赖注入, 在容器中建立bean与bean之间的依赖关系的整个过程。

二者目的是充分解耦

AOP(Aspect Oriented Programming): 面向切面编程, 功能的横向抽取, 主要的实现方式是Proxy

IoC入门案例

2026年3月1日 11:27

①：导入Spring坐标

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context</artifactId>
  <version>5.2.10.RELEASE</version>
</dependency>
```

②：定义Spring管理的类（接口）

```
public interface BookService {
    public void save();
}
```

```
public class BookServiceImpl implements BookService {

    private BookDao bookDao = new BookDaoImpl();

    public void save() {
        bookDao.save();
    }

}
```

③：创建Spring配置文件，配置对应类作为Spring管理的bean

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="bookService" class="com.itheima.service.impl.BookServiceImpl"></bean>
</beans>
```

注意事项

bean定义时id属性在同一个上下文中不能重复

④：初始化IoC容器（Spring核心容器/Spring容器），通过容器获取bean

```
public class App {  
    public static void main(String[] args) {  
        //加载配置文件得到上下文对象，也就是容器对象  
        ApplicationContext ctx = new ClassPathXmlApplicationContext("applicationContext.xml");  
        //获取资源  
        BookService bookService = (BookService) ctx.getBean("bookService");  
        bookService.save();  
    }  
}
```

整合Mybatis，固定格式，只需改包名，类名一般为

MybatisConfig

- 整合MyBatis

```
<configuration>  
    <properties resource="jdbc.properties"></properties>  
    <typeAliases>  
        <package name="com.itheima.domain"/>  
    </typeAliases>  
    <environments default="mysql">  
        <environment id="mysql">  
            <transactionManager type="JDBC"></transactionManager>  
            <dataSource type="POOLED">  
                <property name="driver" value="${jdbc.driver}"></property>  
                <property name="url" value="${jdbc.url}"></property>  
                <property name="username" value="${jdbc.username}"></property>  
                <property name="password" value="${jdbc.password}"></property>  
            </dataSource>  
        </environment>  
    </environments>  
    <mappers>  
        <package name="com.itheima.dao"></package>  
    </mappers>  
</configuration>
```

@Bean

```
public SqlSessionFactoryBean sqlSessionFactory(DataSource dataSource){  
    SqlSessionFactoryBean ssfb = new SqlSessionFactoryBean();  
    ssfb.setTypeAliasesPackage("com.itheima.domain");  
    ssfb.setDataSource(dataSource);  
    return ssfb;  
}
```

```
<configuration>  
    <properties resource="jdbc.properties"></properties>  
    <typeAliases>  
        <package name="com.itheima.domain"/>  
    </typeAliases>  
    <environments default="mysql">  
        <environment id="mysql">  
            <transactionManager type="JDBC"></transactionManager>  
            <dataSource type="POOLED">  
                <property name="driver" value="${jdbc.driver}"></property>  
                <property name="url" value="${jdbc.url}"></property>  
                <property name="username" value="${jdbc.username}"></property>  
                <property name="password" value="${jdbc.password}"></property>  
            </dataSource>  
        </environment>  
    </environments>  
    <mappers>  
        <package name="com.itheima.dao"></package>  
    </mappers>  
</configuration>
```

@Bean

```
public MapperScannerConfigurer mapperScannerConfigurer(){  
    MapperScannerConfigurer msc = new MapperScannerConfigurer();  
    msc.setBasePackage("com.itheima.dao");  
    return msc;  
}
```

整合JUnit

- 使用Spring整合JUnit专用的类加载器

```
@RunWith(SpringJUnit4ClassRunner.class)
@ContextConfiguration(classes = SpringConfig.class)
public class BookServiceTest {

    @Autowired
    private BookService bookService;

    @Test
    public void testSave(){
        bookService.save();
    }
}
```

DI入门案例

2026年3月1日 12:41

- 1.取消Service中用new形式创建的对象
- 2.创建set方法提供Service所需的Dao对象

```
//6. 提供对应的set方法  
public void setBookDao(BookDao bookDao) {  
    this.bookDao = bookDao;  
}
```

- 3.配置service和dao之间的关系

```
<!--2. 配置bean-->  
<!--bean 标签标示配置bean  
id属性标示给bean起名字  
class属性标示给bean定义类型-->  
<bean id="bookDao" class="com.itheima.dao.impl.BookDaoImpl"/>  
  
<bean id="bookService" class="com.itheima.service.impl.BookServiceImpl">  
    <!--7. 配置server 与dao的关系-->  
    <!--property 标签表示配置当前bean的属性  
    name属性表示配置哪一个具体的属性  
    ref属性表示参照哪一个bean-->  
    <property name="bookDao" ref="bookDao"/>  
</bean>
```

ref (引用) 对应的是bean id

传递值的属性为value

正在看

I bean作用范围配置

类别	描述
名称	scope
类型	属性
所属	bean标签
功能	定义bean的作用范围，可选范围如下 ● singleton：单例（默认） ● prototype：非单例
范例	<code><bean id="bookDao" class="com.itheima.dao.impl.BookDaoImpl" scope="prototype" /></code>

实例化bean

- 1.spring创建bean调用无参的构造方法
- 2.静态工厂创造
- 3.示例工厂初始化bean

```
public class UserDaoFactory {  
    public UserDao getUserDao(){  
        return new UserDaoImpl();  
    }  
}
```

Xml配置

```
<!-- 方式三：使用实例工厂实例化bean-->  
<bean id="userFactory" class="com.itheima.factory.UserDaoFactory"/>  
  
<bean id="userDao" factory-method="getUserDao" factory-bean="userFactory"/>
```

```
// 创建实例工厂对象  
UserDaoFactory userDaoFactory = new UserDaoFactory();  
// 通过实例工厂对象创建对象  
UserDao userDao = userDaoFactory.getUserDao();  
userDao.save();
```

4.FactoryBean方法

xml配置

```
<!-- 方式四：使用FactoryBean实例化bean-->  
<bean id="userDao" class="com.itheima.factory.UserDaoFactoryBean"/>
```


//代替原始实例工厂中创建对象的方法

```
public UserDao getObject() throws Exception {  
    return new UserDaoImpl();  
}  
  
public Class<?> getObjectType() {  
    return UserDao.class;  
}
```

Bean的生命周期

bean生命周期

- 初始化容器
 1. 创建对象（内存分配）
 2. 执行构造方法
 3. 执行属性注入（set操作）
 4. 执行bean初始化方法
- 使用bean
 1. 执行业务操作
- 关闭/销毁容器
 1. 执行bean销毁方法

bean实例化后开始，容器关闭后结束

- 提供生命周期控制方法

```
public class BookDaoImpl implements BookDao {  
    public void save() {  
        System.out.println("book dao save ...");  
    }  
    public void init(){  
        System.out.println("book init ...");  
    }  
    public void destroy(){  
        System.out.println("book destroy ...");  
    }  
}
```

- 配置生命周期控制方法

```
<bean id="bookDao" class="com.itheima.dao.impl.BookDaoImpl" init-method="init" destroy-method="destroy"/>
```

- 实现InitializingBean, DisposableBean接口

```
public class BookServiceImpl implements BookService , InitializingBean, DisposableBean {  
    public void save() {  
        System.out.println("book service save ...");  
    }  
    public void afterPropertiesSet() throws Exception {  
        System.out.println("afterPropertiesSet");  
    }  
    public void destroy() throws Exception {  
        System.out.println("destroy");  
    }  
}
```

- 容器关闭前触发bean的销毁
- 关闭容器方式：
 - 手工关闭容器
ConfigurableApplicationContext接口close()操作
 - 注册关闭钩子，在虚拟机退出前先关闭容器再退出虚拟机
ConfigurableApplicationContext接口registerShutdownHook()操作

依赖注入方法

2026年3月1日 16:26

1.set注入（参考DI入门案例）

2.构造方法注入（使用constructor-arg标签）可通过传递位置或参数位置方法解耦配置与参数

- 在bean中定义引用类型属性并提供可访问的**构造方法**

```
public class BookServiceImpl implements BookService{  
    private BookDao bookDao;  
    public BookServiceImpl(BookDao bookDao) {  
        this.bookDao = bookDao;  
    }  
}
```

- 配置中使用**constructor-arg**标签**ref**属性注入引用类型对象

```
<bean id="bookService" class="com.itheima.service.impl.BookServiceImpl">  
    <constructor-arg name="bookDao" ref="bookDao"/>  
</bean>  
<bean id="bookDao" class="com.itheima.dao.impl.BookDaoImpl"/>
```

依赖注入方式选择

- 强制依赖使用构造器进行，使用setter注入有概率不进行注入导致null对象出现
- 可选依赖使用setter注入进行，灵活性强
- Spring框架倡导使用构造器，第三方框架内部大多数采用构造器注入的形式进行数据初始化，相对严谨
- 如果有必要可以两者同时使用，使用构造器注入完成强制依赖的注入，使用setter注入完成可选依赖的注入
- 实际开发过程中还要根据实际情况分析，如果受控对象没有提供setter方法就必须使用构造器注入
- 自己开发的模块推荐使用setter注入**

依赖自动装配

IoC容器根据bean所依赖的资源在容器中自动查找并注入到bean中的过程
在bean标签中使用autowire属性值为byType、byName、

特征

- 自动装配用于引用类型依赖注入，不能对简单类型进行操作
- 使用按类型装配时（byType）必须保障容器中相同类型的bean唯一，推荐使用
- 使用按名称装配时（byName）必须保障容器中具有指定名称的bean，因变量名与配置耦合，不推荐使用
- 自动装配优先级低于setter注入与构造器注入，同时出现时自动装配配置失效

集合注入

2026年3月2日 16:31

- 注入数组对象

```
<property name="array">
  <array>
    <value>100</value>
    <value>200</value>
    <value>300</value>
  </array>
</property>
```

- 注入List对象 (重点)

```
<property name="list">
  <list>
    <value>itcast</value>
    <value>itheima</value>
    <value>boxuegu</value>
  </list>
</property>
```

- 注入Set对象

```
<property name="set">
  <set>
    <value>itcast</value>
    <value>itheima</value>
    <value>boxuegu</value>
  </set>
</property>
```

- 注入Map对象 (重点)

```
<property name="map">
  <map>
    <entry key="country" value="china"/>
    <entry key="province" value="henan"/>
    <entry key="city" value="kaifeng"/>
  </map>
</property>
```

- 注入Properties对象

```
<property name="properties">
  <props>
    <prop key="country">china</prop>
    <prop key="province">henan</prop>
    <prop key="city">kaifeng</prop>
  </props>
</property>
```

加载properties文件

2026年3月2日 16:38

1. 开一个新的名为context的命名空间

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.springframework.org/schema/context
           http://www.springframework.org/schema/context/spring-context.xsd"
>
    <!-- 管理DruidDataSource对象 -->
```

2. 使用context空间加载properties文件，然后使用\${}占位符读取文件中的属性

```
<context:property-placeholder location="jdbc.properties"/>

<bean class="com.alibaba.druid.pool.DruidDataSource">
    <property name="driverClassName" value="${jdbc.driver}"/>
    <property name="url" value="${}"/>
    <property name="username" value="${}"/>
    <property name="password" value="${}"/>
</bean>
```

jdbc.properties的内容

```
jdbc.driver=com.mysql.jdbc.Driver
jdbc.url=jdbc:mysql://127.0.0.1:3306/spring_db
jdbc.username=root
jdbc.password=root
```

加载所有的properties文件location="classpath*:*.properties"

- 不加载系统属性

```
<context:property-placeholder location="jdbc.properties" system-properties-mode="NEVER"/>
```

- 加载多个properties文件

```
<context:property-placeholder location="jdbc.properties,msg.properties"/>
```

- 加载所有properties文件

```
<context:property-placeholder location="*.properties"/>
```

- 加载properties文件**标准格式**

```
<context:property-placeholder location="classpath:*.properties"/>
```

- 从类路径或jar包中搜索并加载properties文件

```
<context:property-placeholder location="classpath*:*.properties"/>
```

容器

2026年3月2日 17:21

- 方式一：类路径加载配置文件

```
ApplicationContext ctx = new ClassPathXmlApplicationContext("applicationContext.xml");
```

- 方式二：文件路径加载配置文件

```
ApplicationContext ctx = new FileSystemXmlApplicationContext("D:\\applicationContext.xml");
```

- 加载多个配置文件

```
ApplicationContext ctx = new ClassPathXmlApplicationContext("bean1.xml", "bean2.xml");
```

- 方式一：使用bean名称获取

```
BookDao bookDao = (BookDao) ctx.getBean("bookDao");
```

- 方式二：使用bean名称获取并指定类型

```
BookDao bookDao = ctx.getBean("bookDao", BookDao.class);
```

- 方式三：使用bean类型获取

```
BookDao bookDao = ctx.getBean(BookDao.class);
```

注解开发模式

2026年3月2日 19:20

- 使用@Component定义bean

```
@Component("bookDao")
public class BookDaoImpl implements BookDao {
}

@Component
public class BookServiceImpl implements BookService {
}
```

- 核心配置文件中通过组件扫描加载bean

```
<context:component-scan base-package="com.itheima"/>
```

- Spring提供@Component注解的三个衍生注解
 - @Controller：用于表现层bean定义
 - @Service：用于业务层bean定义
 - @Repository：用于数据层bean定义

纯注解开发

- Spring3.0开启了纯注解开发模式，使用Java类替代配置文件，开启了Spring快速开发赛道
- Java类代替Spring核心配置文件，

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd">
    <context:component-scan base-package="com.itheima"/>
</beans>
```

```
@Configuration
@ComponentScan("com.itheima")
public class SpringConfig {
}
```

- @Configuration注解用于设定当前类为配置类
- @ComponentScan注解用于设定扫描路径，此注解只能添加一次，多个数据请用数组格式

```
@ComponentScan({com.itheima.service,"com.itheima.dao"})
```

- 使用@PostConstruct、@PreDestroy定义bean生命周期

```
@Repository
@Scope("singleton")
public class BookDaoImpl implements BookDao {
    public BookDaoImpl() {
        System.out.println("book dao constructor ...");
    }
    @PostConstruct
    public void init(){
        System.out.println("book init ...");
    }
    @PreDestroy
    public void destroy(){
        System.out.println("book destory ...");
    }
}
```

scope内定义bean单例或者多例

- 使用@Autowired注解开启自动装配模式（按类型）

```
@Service
public class BookServiceImpl implements BookService {
    @Autowired
    private BookDao bookDao;
    public void setBookDao(BookDao bookDao) {
        this.bookDao = bookDao;
    }
    public void save() {
        System.out.println("book service save ...");
        bookDao.save();
    }
}
```

- 注意：自动装配基于反射设计创建对象并暴力反射对应属性为私有属性初始化数据，因此无需提供setter方法
- 注意：自动装配建议使用无参构造方法创建对象（默认），如果不提供对应构造方法，请提供唯一的构造方法

- 使用@Qualifier注解开启指定名称装配bean

```
@Service
public class BookServiceImpl implements BookService {
    @Autowired
    @Qualifier("bookDao")
    private BookDao bookDao;
}
```

- 注意：@Qualifier注解无法单独使用，必须配合@Autowired注解使用

使用@Value("")注入简单值

配置类中导入配置文件PropertySource("文件名") 多文件用数组格式

- 使用@Bean配置第三方bean

```
@Configuration
public class SpringConfig {

    @Bean
    public DataSource dataSource(){
        DruidDataSource ds = new DruidDataSource();
        ds.setDriverClassName("com.mysql.jdbc.Driver");
        ds.setUrl("jdbc:mysql://localhost:3306/spring_db");
        ds.setUsername("root");
        ds.setPassword("root");
        return ds;
    }
}
```

在独立的第三方配置类中配置，再用@Import导入到核心配置

```
public class JdbcConfig {

    @Bean
    public DataSource dataSource(){
        DruidDataSource ds = new DruidDataSource();
        //相关配置
        return ds;
    }
}
```

- 使用@Import注解手动加入配置类到核心配置，此注解只能添加一次，多个数据请用数组格式

```
@Configuration
@Import(JdbcConfig.class)
public class SpringConfig {

}
```

第三方配置类内容

- 简单类型依赖注入

```
public class JdbcConfig {
    @Value("com.mysql.jdbc.Driver")
    private String driver;
    @Value("jdbc:mysql://localhost:3306/spring_db")
    private String url;
    @Value("root")
    private String userName;
    @Value("root")
    private String password;
    @Bean
    public DataSource dataSource(){
        DruidDataSource ds = new DruidDataSource();
        ds.setDriverClassName(driver);
        ds.setUrl(url);
        ds.setUsername(userName);
        ds.setPassword(password);
        return ds;
    }
}
```

```
        ds.setPassword(userName);  
        return ds;  
    }  
}
```

- 引用类型依赖注入

```
@Bean  
public DataSource dataSource(BookService bookService){  
    System.out.println(bookService);  
    DruidDataSource ds = new DruidDataSource();  
    //属性设置  
    return ds;  
}
```

- 引用类型注入只需要为bean定义方法设置形参即可，容器会根据类型自动装配对象

AOP

2026年3月3日 18:59

面向切面编程

概念：在不影响原始设计的基础上进行功能增强

- 连接点 (JoinPoint)：程序执行过程中的任意位置，粒度为执行方法、抛出异常、设置变量等
 - 在SpringAOP中，理解为方法的执行
- 切入点 (Pointcut)：匹配连接点的式子
 - 在SpringAOP中，一个切入点可以只描述一个具体方法，也可以匹配多个方法
 - ◆ 一个具体方法：com.itheima.dao包下的BookDao接口中的无形参无返回值的save方法
 - ◆ 匹配多个方法：所有的save方法，所有的get开头的方法，所有以Dao结尾的接口中的任意方法，所有带有一个参数的方法
- 通知 (Advice)：在切入点处执行的操作，也就是共性功能
 - 在SpringAOP中，功能最终以方法的形式呈现
- 通知类：定义通知的类
- 切面 (Aspect)：描述通知与切入点的对应关系

所需坐标spring.aop (context自带)，org.aspectj

①：导入aop相关坐标

```
<dependency>
  <groupId>org.aspectj</groupId>
  <artifactId>aspectjweaver</artifactId>
  <version>1.9.4</version>
</dependency>
```

说明：spring-context坐标依赖spring-aop坐标

```

  > org.springframework:spring-context:5.2.10.RELEASE
  > org.springframework:spring-aop:5.2.10.RELEASE
  > org.springframework:spring-beans:5.2.10.RELEASE
  > org.springframework:spring-core:5.2.10.RELEASE
  > org.springframework:spring-expression:5.2.10.RELEASE
  > org.aspectj:aspectjweaver:1.9.4
```

②：定义dao接口与实现类

```
public interface BookDao {
    public void save();
    public void update();
}
```

```
@Repository
public class BookDaoImpl implements BookDao {
    public void save() {
        System.out.println(System.currentTimeMillis());
        System.out.println("book dao save ...");
    }

    public void update(){
        System.out.println("book dao update ...");
    }
}
```

③：定义通知类，制作通知

```
public class MyAdvice {
    public void before(){
        System.out.println(System.currentTimeMillis());
    }
}
```

④：定义切入点

```
public class MyAdvice {  
    @Pointcut("execution(void com.itheima.dao.BookDao.update())")  
    private void pt(){}  
}
```

说明：切入点定义依托一个不具有实际意义的方法进行，即无参数，无返回值，方法体无实际逻辑

⑤：绑定切入点与通知关系，并指定通知添加到原始连接点的具体执行位置

```
public class MyAdvice {  
    @Pointcut("execution(void com.itheima.dao.BookDao.update())")  
    private void pt(){}  
  
    @Before("pt()")  
    public void before(){  
        System.out.println(System.currentTimeMillis());  
    }  
}
```

⑥：定义通知类受Spring容器管理，并定义当前类为切面类

```
@Component  
@Aspect  
public class MyAdvice {  
    @Pointcut("execution(void com.itheima.dao.BookDao.update())")  
    private void pt(){}  
  
    @Before("pt()")  
    public void before(){  
        System.out.println(System.currentTimeMillis());  
    }  
}
```

⑦：开启Spring对AOP注解驱动支持

```
@Configuration  
@ComponentScan("com.itheima")  
@EnableAspectJAutoProxy  
public class SpringConfig {  
}
```

AOP工作流程

1. Spring容器启动
2. 读取所有切面配置中的切入点
3. 初始化bean，判定bean对应的类中的方法是否匹配到任意切入点
 - 匹配失败，创建对象
 - 匹配成功，创建原始对象（**目标对象**）的**代理对象**
4. 获取bean执行方法
 - 获取bean，调用方法并执行，完成操作
 - 获取的bean是代理对象时，根据代理对象的运行模式运行原始方法与增强的内容，完成操作

AOP的本质是代理模式

- 切入点表达式标准格式：动作关键字（访问修饰符 返回值 包名.类/接口名.方法名（参数）异常名）

```
execution (public User com.itheima.service.UserService.findById (int) )
```

- 动作关键字：描述切入点的行为动作，例如execution表示执行到指定切入点
- 访问修饰符：public，private等，可以省略
- 返回值
- 包名
- 类/接口名
- 方法名
- 参数
- 异常名：方法定义中抛出指定异常，可以省略

- 可以使用通配符描述切入点，快速描述

- *：单个独立的任意符号，可以独立出现，也可以作为前缀或者后缀的匹配符出现

```
execution (public * com.itheima.*.UserService.find* (*) )
```

匹配com.itheima包下的任意包中的UserService类或接口中所有find开头的带有一个参数的方法

- ..：多个连续的任意符号，可以独立出现，常用于简化包名与参数的书写

```
execution (public User com..UserService.findById (..) )
```

匹配com包下的任意包中的UserService类或接口中所有名称为findById的方法

- +：专用于匹配子类类型

```
execution(* *..*Service+.*(..))
```

- 书写技巧

- 所有代码按照标准规范开发，否则以下技巧全部失效
- 描述切入点**通常描述接口**，而不描述实现类
- 访问控制修饰符针对接口开发均采用public描述（**可省略访问控制修饰符描述**）
- 返回值类型对于增删改类使用精准类型加速匹配，对于查询类使用*通配快速描述
- **包名书写尽量不使用.匹配**，效率过低，常用*做单个包描述匹配，或精准匹配
- **接口名/类名书写名称与模块相关的采用*匹配**，例如UserService书写成*Service，绑定业务层接口名
- **方法名书写以动词进行精准匹配**，名词采用*匹配，例如getById书写成getBy*,selectAll书写成selectAll
- 参数规则较为复杂，根据业务方法灵活调整
- 通常**不使用异常**作为**匹配规则**

- 名称：@Around（重点，常用）

- 类型：**方法注解**

- 位置：通知方法定义上方

- 作用：设置当前通知方法与切入点之间的绑定关系，当前通知方法在原始切入点方法前后运行

- 范例：

```
@Around("pt()")
public Object around(ProceedingJoinPoint pjp) throws Throwable {
    System.out.println("around before advice ...");
    Object ret = pjp.proceed();
    System.out.println("around after advice ...");
    return ret;
}
```


● @Around注意事项

1. 环绕通知必须依赖形参ProceedingJoinPoint才能实现对原始方法的调用，进而实现原始方法调用前后同时添加通知
2. 通知中如果未使用ProceedingJoinPoint对原始方法进行调用将跳过原始方法的执行
3. 对原始方法的调用可以不接收返回值，通知方法设置成void即可，如果接收返回值，必须设定为Object类型
4. 原始方法的返回值如果是void类型，通知方法的返回值类型可以设置成void，也可以设置成Object
5. 由于无法预知原始方法运行后是否会抛出异常，因此环绕通知方法必须抛出Throwable对象

spring事务管理

2026年3月3日 20:24

作用：在事务层或业务层保障同一系列的操作同成功或同失败

①：在业务层接口上添加Spring事务管理

```
public interface AccountService {  
    @Transactional  
    public void transfer(String out,String in ,Double money);  
}
```

注意事项

Spring注解式事务通常添加在业务层接口中而不会添加到业务层实现类中，降低耦合

注解式事务可以添加到业务方法上表示当前方法开启事务，也可以添加到接口上表示当前接口所有方法开启事务

②：设置事务管理器

```
@Bean  
public PlatformTransactionManager transactionManager(DataSource dataSource){  
    DataSourceTransactionManager ptm = new DataSourceTransactionManager();  
    ptm.setDataSource(dataSource);  
    return ptm;  
}
```

③：开启注解式事务驱动

```
@Configuration  
@ComponentScan("com.itheima")  
@PropertySource("classpath:jdbc.properties")  
@Import({JdbcConfig.class,MybatisConfig.class})  
@EnableTransactionManagement  
public class SpringConfig {  
}
```

事务相关配置

属性	作用	示例
readOnly	设置是否为只读事务	readOnly=true 只读事务
timeout	设置事务超时时间	timeout = -1 (永不超时)
rollbackFor	设置事务回滚异常 (class)	rollbackFor = {NullPointerException.class}
rollbackForClassName	设置事务回滚异常 (String)	同上格式为字符串
noRollbackFor	设置事务不回滚异常 (class)	noRollbackFor = {NullPointerException.class}
noRollbackForClassName	设置事务不回滚异常 (String)	同上格式为字符串
propagation	设置事务传播行为	

SpringMVC

2026年3月4日 21:55

一种基于Java实现MVC模型的亲两级Web框架

使用步骤

①：使用SpringMVC技术需要先导入SpringMVC坐标与Servlet坐

```
<dependency>
  <groupId>javax.servlet</groupId>
  <artifactId>javax.servlet-api</artifactId>
  <version>3.1.0</version>
  <scope>provided</scope>
</dependency>
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-webmvc</artifactId>
  <version>5.2.10.RELEASE</version>
</dependency>
```

②：创建SpringMVC控制器类（等同于Servlet功能）

```
@Controller
public class UserController {
    @RequestMapping("/save")
    @ResponseBody
    public String save(){
        System.out.println("user save ...");
        return "{ 'info': 'springmvc' }";
    }
}
```

③：初始化SpringMVC环境（同Spring环境），设定SpringMVC加载对应的bean

```
@Configuration
@ComponentScan("com.itheima.controller")
public class SpringMvcConfig {
}
```

④：初始化Servlet容器，加载SpringMVC环境，并设置SpringMVC技术处理的请求

```
public class ServletContainersInitConfig extends AbstractDispatcherServletInitializer {  
    protected WebApplicationContext createServletApplicationContext() {  
        AnnotationConfigWebApplicationContext ctx = new AnnotationConfigWebApplicationContext();  
        ctx.register(SpringMvcConfig.class);  
        return ctx;  
    }  
    protected String[] getServletMappings() {  
        return new String[]{"/*"};  
    }  
    protected WebApplicationContext createRootApplicationContext() {  
        return null;  
    }  
}
```